

```

@Override
protected void configure(HttpSecurity http) throws Exception {
    http
        .authorizeRequests()
            .antMatchers("/api/tacos", "/orders").hasAuthority("USER")
            .antMatchers("/**").permitAll();
}
}

```

Now let's see what this same configuration might look like for a reactive Spring Web-Flux application. The following listing shows a reactive security configuration class that's roughly equivalent to the simple security configuration from before.

Listing 12.2 Configuring Spring Security for a Spring WebFlux application

```

@Configuration
@EnableWebFluxSecurity
public class SecurityConfig {

    @Bean
    public SecurityWebFilterChain securityWebFilterChain(
        ServerHttpSecurity http) {

        return http
            .authorizeExchange()
                .pathMatchers("/api/tacos", "/orders").hasAuthority("USER")
                .anyExchange().permitAll()
            .and()
                .build();
    }
}

```

As you can see, there's a lot that's familiar, though, at the same time, much is different. Rather than `@EnableWebSecurity`, this new configuration class is annotated with `@EnableWebFluxSecurity`. What's more, the configuration class doesn't extend `WebSecurityConfigurerAdapter` or any other base class whatsoever. Therefore, it also doesn't override any `configure()` methods.

In place of a `configure()` method, you declare a bean of type `SecurityWebFilterChain` with the `securityWebFilterChain()` method. The body of `securityWebFilterChain()` isn't much different from the previous configuration's `configure()` method, but there are some subtle changes.

Primarily, the configuration is declared using a given `ServerHttpSecurity` object instead of an `HttpSecurity` object. Using the given `ServerHttpSecurity`, you can call `authorizeExchange()`, which is roughly equivalent to `authorizeRequests()`, to declare request-level security.

NOTE `ServerHttpSecurity` is new to Spring Security 5 and is the reactive analog to `HttpSecurity`.

When matching paths, you can still use Ant-style wildcard paths, but do so with the `pathMatchers()` method instead of `antMatchers()`. And as a convenience, you no longer need to specify a catchall Ant-style path of `/**` because the `anyExchange()` returns the catchall you need.

Finally, because you're declaring the `SecurityWebFilterChain` as a bean instead of overriding a framework method, you must call the `build()` method to assemble all of the security rules into the `SecurityWebFilterChain` to be returned.

Aside from those small differences, configuring web security isn't that different for Spring WebFlux than for Spring MVC. But what about user details?

12.5.2 Configuring a reactive user details service

When extending `WebSecurityConfigurerAdapter`, you override one `configure()` method to declare web security rules and another `configure()` method to configure authentication logic, typically by defining a `UserDetails` object. As a reminder of what this looks like, consider the following overridden `configure()` method that uses an injected `UserRepository` object in an anonymous implementation of `UserDetailsService` to look up a user by username:

```
@Autowired
UserRepository userRepo;

@Override
protected void
    configure(AuthenticationManagerBuilder auth)
        throws Exception {
    auth
        .userDetailsService(new UserDetailsService() {
            @Override
            public UserDetails loadUserByUsername(String username)
                throws UsernameNotFoundException {
                User user = userRepo.findByUsername(username);
                if (user == null) {
                    throw new UsernameNotFoundException(
                        username + " not found");
                }
                return user.toUserDetails();
            }
        });
}
```

In this nonreactive configuration, you override the only method required by `UserDetailsService`: `loadUserByUsername()`. Inside of that method, you use the given `UserRepository` to look up the user by the given username. If the name isn't found, you throw a `UsernameNotFoundException`. But if it's found, then you call a helper method, `toUserDetails()`, to return the resulting `UserDetails` object.

In a reactive security configuration, you don't override a `configure()` method. Instead, you declare a `ReactiveUserDetailsService` bean. `ReactiveUserDetailsService` is the reactive equivalent to `UserDetailsService`. Like `UserDetailsService`,

`ReactiveUserDetailsService` requires implementation of only a single method. Specifically, the `findByUsername()` method returns a `Mono<UserDetails>` instead of a raw `UserDetails` object.

In the following example, the `ReactiveUserDetailsService` bean is declared to use a given `UserRepository`, which is presumed to be a reactive Spring Data repository (which we'll talk more about in the next chapter):

```
@Bean
public ReactiveUserDetailsService userDetailsService(
    UserRepository userRepo) {
    return new ReactiveUserDetailsService() {
        @Override
        public Mono<UserDetails> findByUsername(String username) {
            return userRepo.findByUsername(username)
                .map(user -> {
                    return user.toUserDetails();
                });
        }
    };
}
```

Here, a `Mono<UserDetails>` is returned as required, but the `UserRepository.findByUsername()` method returns a `Mono<User>`. Because it's a `Mono`, you can chain operations on it, such as a `map()` operation to map the `Mono<User>` to a `Mono<UserDetails>`.

In this case, the `map()` operation is applied with a lambda that calls the helper `toUserDetails()` method on the `User` object published by the `Mono`. This converts the `User` to a `UserDetails`. As a consequence, the `.map()` operation returns a `Mono<UserDetails>`, which is precisely what the `ReactiveUserDetailsService.findByUsername()` requires. If `findByUsername()` can't find a matching user, then the `Mono` returned will be empty, indicating no match and resulting in a failure to authenticate.

Summary

- Spring WebFlux offers a reactive web framework whose programming model mirrors that of Spring MVC and even shares many of the same annotations.
- Spring also offers a functional programming model as an alternative to Spring WebFlux's annotation-based programming model.
- Reactive controllers can be tested with `WebTestClient`.
- On the client side, Spring offers `WebClient`, a reactive analog to Spring's `RestTemplate`.
- Although WebFlux has some significant implications for the underlying mechanisms for securing a web application, Spring Security 5 supports reactive security with a programming model that isn't dramatically different from nonreactive Spring MVC applications.

13

Persisting data reactively

This chapter covers

- Reactive relational persistence with R2DBC
- Defining reactive repositories for MongoDB and Cassandra
- Testing reactive repositories

If we've learned one thing from science fiction, it's that if you want to improve upon past experiences, all you need is a little time travel. It worked in *Back to the Future*, several episodes of various Star Trek shows, *Avengers: Endgame*, and Stephen King's *11/22/63*. (OK, well maybe that last one didn't turn out better. But you get the idea.)

In this chapter, we're going to rewind back to chapters 3 and 4, revisiting the repositories we created for relational databases, MongoDB, and Cassandra. This time, we're going to improve on them by taking advantage of some of Spring Data's reactive repository support, allowing us to work with those repositories in a non-blocking fashion.

Let's start by looking at Spring Data R2DBC, a reactive alternative to Spring Data JDBC for persistence to relational databases.

13.1 Working with R2DBC

Reactive Relational Database Connectivity, or R2DBC (<https://r2dbc.io/>) as it is commonly known, is a relatively new option for working with relational data using reactive types. It is effectively a reactive alternative to JDBC, enabling nonblocking persistence against conventional relational databases such as MySQL, PostgreSQL, H2, and Oracle. Because it's built on Reactive Streams, it is quite different from JDBC and is a separate specification, unrelated to Java SE.

Spring Data R2DBC is a subproject of Spring Data that offers automatic repository support for R2DBC, much the same as Spring Data JDBC, which we looked at in chapter 3. Unlike Spring Data JDBC, however, Spring Data R2DBC doesn't require strict adherence to domain-driven design concepts. In fact, as you'll soon see, attempting to persist data through an aggregate root requires a bit more work with Spring Data R2DBC than with Spring Data JDBC.

To use Spring Data R2DBC, you'll need to add a starter dependency to your project's build. For a Maven-built project, the dependency looks like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-r2dbc</artifactId>
</dependency>
```

Or, if you're using the Initializr, select the Spring Data R2DBC check box when creating your project.

You'll also need a relational database to persist data to, along with a corresponding R2DBC driver. For our project, we'll be using an in-memory H2 database. Therefore, we need to add two dependencies: the H2 database library itself and the H2 R2DBC driver. The Maven dependencies follow:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.r2dbc</groupId>
  <artifactId>r2dbc-h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

If you're using a different database, then you'll need to add the corresponding R2DBC driver dependency for the database of your choice.

Now that the dependencies are in place, let's see how Spring Data R2DBC works. Let's start by defining the domain entities.

13.1.1 Defining domain entities for R2DBC

To get to know Spring Data R2DBC, we'll recreate just the persistence layer of the Taco Cloud application, focusing only on the components that are necessary for persisting taco and order data. This includes creating domain entities for `TacoOrder`, `Taco`, and `Ingredient`, along with corresponding repositories for each.

The first domain entity class we'll create is the `Ingredient` class. It will look something like the next code listing.

Listing 13.1 The `Ingredient` entity class for R2DBC persistence

```
package tacos;

import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.EqualsAndHashCode;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
@EqualsAndHashCode(exclude = "id")
public class Ingredient {

    @Id
    private Long id;

    private @NonNull String slug;

    private @NonNull String name;
    private @NonNull Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

As you can see, this isn't much different from other incarnations of the `Ingredient` class that we've created before. Note the following two noteworthy differences:

- Spring Data R2DBC requires that properties have setter methods, so rather than define most properties as `final`, they have to be non-`final`. But to help Lombok create a required arguments constructor, we annotate most of the properties with `@NonNull`. This will cause Lombok and the `@RequiredArgsConstructor` annotation to include those properties in the constructor.
- When saving an object through a Spring Data R2DBC repository, if the object's ID property is non-null, it is treated as an update. In the case of `Ingredient`, the `id` property was previously typed as `String` and specified at

creation time. But doing that with Spring Data R2DBC results in an error. So, here we shift that String ID to a new property named `slug`, which is just a pseudo-ID for the `Ingredient`, and use a Long ID property with a value generated by the database.

The corresponding database table is defined in `schema.sql` like this:

```
create table Ingredient (
    id identity,
    slug varchar(4) not null,
    name varchar(25) not null,
    type varchar(10) not null
);
```

The `Taco` entity class is also quite similar to its Spring Data JDBC counterpart, as shown in the next code.

Listing 13.2 The `Taco` entity class for R2DBC persistence

```
package tacos;

import java.util.HashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.NonNull;
import lombok.RequiredArgsConstructor;

@Data
@NoArgsConstructor
@RequiredArgsConstructor
public class Taco {

    @Id
    private Long id;

    private @NonNull String name;

    private Set<Long> ingredientIds = new HashSet<>();

    public void addIngredient(Ingredient ingredient) {
        ingredientIds.add(ingredient.getId());
    }
}
```

As with the `Ingredient` class, we have to allow for setter methods on the entity's fields, thus the use of `@NonNull` instead of `final`.

But what's especially interesting here is that instead of having a collection of `Ingredient` objects, `Taco` has a `Set<Long>` referencing the IDs of `Ingredient` objects

that are part of this taco. Set was chosen over List to guarantee uniqueness. But why must we use a Set<Long> and not a Set<Ingredient> for the ingredient collection?

Unlike other Spring Data projects, Spring Data R2DBC doesn't currently support direct relationships between entities (at least not at this time). As a relatively new project, Spring Data R2DBC is still working through some of the challenges of handling relationships in a nonblocking way. This may change in future versions of Spring Data R2DBC.

Until then, we can't have Taco referencing a collection of Ingredient and expect persistence to just work. Instead, we have the following options when it comes to dealing with relationships:

- *Define entities with references to the IDs of related objects.* In this case, the corresponding column in the database table must be defined with an array type, if possible. H2 and PostgreSQL are two databases that support array columns, but many others do not. Also, even if the database supports array columns, it may not be possible to define the entries as foreign keys to the referenced table, making it impossible to enforce referential integrity.
- *Define entities and their corresponding tables to match each other perfectly.* For collections, this would mean that the referred object would have a column mapping back to the referring table. For example, the table for Taco objects would need to have a column that points back to the TacoOrder that the Taco is a part of.
- *Serialize referenced entities to JSON and store the JSON in a large VARCHAR column.* This works especially well if there's no need to query through to the referenced objects. It does, however, have potential limits to how big the JSON-serialized object(s) can be due to limits to the length of the corresponding VARCHAR column. Moreover, we won't have any way to leverage the database schema to guarantee referential integrity, because the referenced objects will be stored as a simple string value (which could contain anything).

Although none of these options are ideal, after weighing them, we'll choose the first option for the Taco object. The Taco class has a Set<Long> that references one or more Ingredient IDs. This means that the corresponding table must have an array column to store those IDs. For the H2 database, the Taco table is defined like this:

```
create table Taco (  
    id identity,  
    name varchar(50) not null,  
    ingredient_ids array  
);
```

The array type used on the ingredient_ids column is specific to H2. For PostgreSQL, that column might be defined as integer[]. Consult your chosen database documentation for details on how to define array columns. Note that not all database implementations support array columns, so you may need to choose one of the other options for modeling relationships.

Finally, the `TacoOrder` class, as shown in the next listing, is defined using many of the things we've already employed in defining our domain entities for persistence with Spring Data R2DBC.

Listing 13.3 The `TacoOrder` entity class for R2DBC persistence

```
package tacos;

import java.util.LinkedHashSet;
import java.util.Set;
import org.springframework.data.annotation.Id;
import lombok.Data;

@Data
public class TacoOrder {

    @Id
    private Long id;

    private String deliveryName;
    private String deliveryStreet;
    private String deliveryCity;
    private String deliveryState;
    private String deliveryZip;
    private String ccNumber;
    private String ccExpiration;
    private String ccCVV;

    private Set<Long> tacoIds = new LinkedHashSet<>();

    private List<Taco> tacos = new ArrayList<>();
    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}
```

As you can see, aside from having a few more properties, the `TacoOrder` class follows the same pattern as the `Taco` class. It references its child `Taco` objects via a `Set<Long>`. A little later, though, we'll see how to get complete `Taco` objects into a `TacoOrder`, even though Spring Data R2DBC doesn't directly support relationships in that way.

The database schema for the `Taco_Order` table looks like this:

```
create table Taco_Order (
    id identity,
    delivery_name varchar(50) not null,
    delivery_street varchar(50) not null,
    delivery_city varchar(50) not null,
    delivery_state varchar(2) not null,
    delivery_zip varchar(10) not null,
    cc_number varchar(16) not null,
    cc_expiration varchar(5) not null,
```

```
cc_cvv varchar(3) not null,
taco_ids array
);
```

Just like the Taco table, which references ingredients with an array column, the Taco-Order table references its child Tacos with a `taco_ids` column defined as an array column. Again, this schema is for an H2 database; consult your database documentation for details on support and creation of array columns.

Oftentimes, a production application already has its schema defined through other means, and such scripts aren't desirable except for tests. Therefore, this bean is defined in a configuration that is loaded only when running automated tests and isn't available in the runtime application context. We'll see an example of such a test for testing R2DBC repositories after we have defined those services.

What's more, notice that this bean uses only the `schema.sql` file from the root of the classpath (under `src/main/resources` in the project). If you'd like other SQL scripts to be included as part of the database initialization, add more `ResourceDatabasePopulator` objects in the call to `populator.addPopulators()`.

Now that we've defined our entities and their corresponding database schemas, let's create the repositories through which we'll save and fetch taco data.

13.1.2 Defining reactive repositories

In chapters 3 and 4, we defined our repositories as interfaces that extend Spring Data's `CrudRepository` interface. But that base repository interface dealt with singular objects and `Iterable` collections. In contrast, we'd expect that a reactive repository would deal in `Mono` and `Flux` objects.

That's why Spring Data offers `ReactiveCrudRepository` for defining reactive repositories. `ReactiveCrudRepository` operates very much like `CrudRepository`. To create a repository, define an interface that extends `ReactiveCrudRepository`, such as this:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, Long> {
}
```

On the surface, the only difference between this `OrderRepository` and the ones we defined in chapters 3 and 4 is that it extends `ReactiveCrudRepository` instead of `CrudRepository`. But what's significantly different is that its methods return `Mono` and `Flux` types instead of a single `TacoOrder` or `Iterable<TacoOrder>`. Two examples include the `findById()` method, which returns a `Mono<TacoOrder>`, and `findAll()`, which returns a `Flux<TacoOrder>`.

To see how this reactive repository might work in action, suppose that you want to fetch all `TacoOrder` objects and print their delivery names to standard output. In that case, you might write some code like the next snippet.

Listing 13.4 Calling a reactive repository method

```
@Autowired
OrderRepository orderRepo;

...

orderRepository.findAll()
    .doOnNext(order -> {
        System.out.println(
            "Deliver to: " + order.getDeliveryName());
    })
    .subscribe();
```

Here, the call to `findAll()` returns a `Flux<TacoOrder>` on which we have added a `doOnNext()` to print the delivery name. Finally, the call to `subscribe()` kicks off the flow of data through the `Flux`.

In the Spring Data JDBC example from chapter 3, `TacoOrder` was the aggregate root, with `Taco` being a child in that aggregate. Therefore, `Taco` objects were persisted as part of a `TacoOrder`, and there was no need to define a repository dedicated to `Taco` persistence. But Spring Data R2DBC doesn't support proper aggregate roots this way, so we'll need a `TacoRepository` through which `Taco` objects are persisted. See the next listing for such a repository.

Listing 13.5 Persisting Taco objects with a reactive repository

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import tacos.Taco;

public interface TacoRepository
    extends ReactiveCrudRepository<Taco, Long> {
}

}
```

As you can see, `TacoRepository` isn't much different from `OrderRepository`. It extends `ReactiveCrudRepository` to give us reactive types when working with `Taco` persistence. There aren't many surprises here.

On the other hand, `IngredientRepository` is slightly more interesting, as shown next.

Listing 13.6 Persisting Ingredient objects with a reactive repository

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
```

```
import reactor.core.publisher.Mono;
import tacos.Ingredient;

public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, Long> {

    Mono<Ingredient> findBySlug(String slug);

}
```

As with our other two reactive repositories, `IngredientRepository` extends `ReactiveCrudRepository`. But because we might need a way to look up `Ingredient` objects based on a slug value, `IngredientRepository` includes a `findBySlug()` method that returns a `Mono<Ingredient>`.¹

Now let's see how to write tests to verify that our repositories work.

13.1.3 Testing R2DBC repositories

Spring Data R2DBC includes support for writing integration tests for R2DBC repositories. Specifically, the `@DataR2dbcTest` annotation, when placed on a test class, causes Spring to create an application context with the generated Spring Data R2DBC repositories as beans that can be injected into the test class. Along with `StepVerifier`, which we've used in previous chapters, this enables us to write automated tests against all of the repositories we've created.

For the sake of brevity, we'll focus solely on a single test class: `IngredientRepositoryTest`. This will test `IngredientRepository`, verifying that it can save `Ingredient` objects, fetch a single `Ingredient`, and fetch all saved `Ingredient` objects. The next code sample shows this test class.

Listing 13.7 Testing a Spring Data R2DBC repository

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;

import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;
```

¹ This method wasn't necessary in chapter 3's JDBC-based repository because we were able to have the `id` field serve double duty as both an ID and a slug.

```

@DataR2dbcTest
public class IngredientRepositoryTest {

    @Autowired
    IngredientRepository ingredientRepo;

    @BeforeEach
    public void setup() {
        Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )))

        StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

        StepVerifier.create(ingredientRepo.findAll())
            .recordWith(ArrayList::new)
            .thenConsumeWhile(x -> true)
            .consumeRecordedWith(ingredients -> {
                assertThat(ingredients).hasSize(3);
                assertThat(ingredients).contains(
                    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
                assertThat(ingredients).contains(
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
                assertThat(ingredients).contains(
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
            })
            .verifyComplete();

        StepVerifier.create(ingredientRepo.findBySlug("FLTO"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLTO", "Flour Tortilla",
                    Type.WRAP));
            });
    }
}

```

The `shouldSaveAndFetchIngredients()` method starts by creating a Flux of test Ingredient objects. From this Flux, it uses the `flatMap()` operation to save each Ingredient via the `save()` method on the injected `IngredientRepository`. The call to `subscribe()` opens the flow of data through the Flux, resulting in the Ingredient objects being saved.

Next up, a `StepVerifier` is created from the `Mono<Ingredient>` returned by the repository's `findBySlug()` method. The `Mono<Ingredient>` should have a single `Ingredient` in it, which is what the `assertNext()` method verifies, matching it up with the values expected for an `Ingredient` whose slug is "FLTO". Then, it verifies that the `Mono` is complete.

Finally, another `StepVerifier` is created from the `Flux<Ingredient>` returned by the repository's `findAll()` method. One at a time, it asserts that each `Ingredient` that flows from that `Flux` matches up with the three `Ingredient` objects originally saved at the start of the test method. And, as with the other `StepVerifier`, a call to `verifyComplete()` verifies that the `Mono` is complete and has no more `Ingredient` objects coming through.

Although we focused only on testing the `IngredientRepository`, the same techniques can be used to test any Spring Data R2DBC-generated repository.

So far, so good. We now have defined our domain types and their respective repositories. And we've written a test to verify that they work. We can use them as is if we like. But these repositories make persistence of a `TacoOrder` inconvenient in that we must first create and persist `Taco` objects that are part of that order and then persist the `TacoOrder` object that references the child `Taco` objects. And when reading the `TacoOrder`, we'll receive only a collection of `Taco` IDs and not fully defined `Taco` objects.

It would be nice if we could persist `TacoOrder` as an aggregate root and have its child `Taco` objects be persisted along with it. Likewise, it would be great if we could fetch a `TacoOrder` and have it fully defined with complete `Taco` objects and not just the IDs. Let's define a service-level class that sits in front of `OrderRepository` and `TacoRepository` to mimic the persistence behavior of chapter 3's `OrderRepository`.

13.1.4 Defining an `OrderRepository` aggregate root service

The first step toward persisting `TacoOrder` and `Taco` objects together such that `TacoOrder` is the aggregate root is to add a `Taco` collection property to the `TacoOrder` class. This is shown next.

Listing 13.8 Adding a `Taco` collection to `TacoOrder`

```
@Data
public class TacoOrder {

    ...

    @Transient
    private transient List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
        if (taco.getId() != null) {
            this.tacoIds.add(taco.getId());
        }
    }
}
```

Aside from adding a new `List<Taco>` property named `tacos` to the `TacoOrder` class, the `addTaco()` method now adds the given `Taco` to that list (as well as adding its `id` to the `tacoIds` set as before).

Notice, however, that the `tacos` property is annotated with `@Transient` (as well as marked with Java's `transient` keyword). This indicates that Spring Data R2DBC shouldn't attempt to persist this property. Without the `@Transient` annotation, Spring Data R2DBC would try to persist it and result in an error, due to it not supporting such relationships.

When a `TacoOrder` is saved, only the `tacoIds` property will be written to the database, and the `tacos` property will be ignored. Even so, at least now `TacoOrder` has a place to hold `Taco` objects. That will come in handy both for saving `Taco` objects when a `TacoOrder` is saved and also to read in `Taco` objects when a `TacoOrder` is fetched.

Now we can create a service bean that saves and reads `TacoOrder` objects along with their respective `Taco` objects. Let's start with saving a `TacoOrder`. The `TacoOrderAggregateService` class defined in the next code listing has a `save()` method that does precisely that.

Listing 13.9 Saving TacoOrders and Tacos as an aggregate

```
package tacos.web.api;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;
import lombok.RequiredArgsConstructor;
import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@Service
@RequiredArgsConstructor
public class TacoOrderAggregateService {

    private final TacoRepository tacoRepo;
    private final OrderRepository orderRepo;

    public Mono<TacoOrder> save(TacoOrder tacoOrder) {
        return Mono.just(tacoOrder)
            .flatMap(order -> {
                List<Taco> tacos = order.getTacos();
                order.setTacos(new ArrayList<>());
                return tacoRepo.saveAll(tacos)
                    .map(taco -> {
                        order.addTaco(taco);
                        return order;
                    })
                    .last();
            });
    }
}
```

```

    })
    .flatMap(orderRepo::save);
}

}

```

Although there aren't many lines in listing 13.9, there's a lot going on in the `save()` method that requires some explanation. Firstly, the `TacoOrder` that is received as a parameter is wrapped in a `Mono` using the `Mono.just()` method. This allows us to work with it as a reactive type throughout the rest of the `save()` method.

The next thing we do is apply a `flatMap()` to the `Mono<TacoOrder>` we just created. Both `map()` and `flatMap()` are options for doing transformations on a data object passing through a `Mono` or `Flux`, but because the operations we perform in the course of doing the transformation will result in a `Mono<TacoOrder>`, the `flatMap()` operation ensures that we continue working with a `Mono<TacoOrder>` after the mapping and not a `Mono<Mono<TacoOrder>>`, as would be the case if we used `map()` instead.

The purpose of the mapping is to ensure that the `TacoOrder` ends up with the IDs of the child `Taco` objects and saves those `Taco` objects along the way. Each `Taco` object's ID is probably null initially for a new `TacoOrder`, and we won't know the IDs until after the `Taco` objects have been saved.

After fetching the `List<Taco>` from the `TacoOrder`, which we'll use when saving `Taco` objects, we reset the `tacos` property to an empty list. We'll be rebuilding that list with new `Taco` objects that have been assigned IDs after having been saved.

A call to the `saveAll()` method on the injected `TacoRepository` saves all of our `Taco` objects. The `saveAll()` method returns a `Flux<Taco>` that we then cycle through by way of the `map()` method. In this case, the transformation operation is secondary to the fact that each `Taco` object is being added back to the `TacoOrder`. But to ensure that it's a `TacoOrder` and not a `Taco` that ends up on the resulting `Flux`, the mapping operation returns the `TacoOrder` instead of the `Taco`. A call to `last()` ensures that we won't have duplicate `TacoOrder` objects (one for each `Taco`) as a result of the mapping operation.

At this point, all `Taco` objects should have been saved and then pushed back into the parent `TacoOrder` object, along with their newly assigned IDs. All that's left is to save the `TacoOrder`, which is what the final `flatMap()` call does. Again, we choose `flatMap()` here to ensure that the `Mono<TacoOrder>` returned from the call to `OrderRepository.save()` doesn't get wrapped in another `Mono`. We want our `save()` method to return a `Mono<TacoOrder>`, not a `Mono<Mono<TacoOrder>>`.

Now let's have a look at a method that will read a `TacoOrder` by its ID, reconstituting all of the child `Taco` objects. The following code sample shows a new `findById()` method for that purpose.

Listing 13.10 Reading TacoOrders and Tacos as an aggregate

```

public Mono<TacoOrder> findById(Long id) {
    return orderRepo

```



```

        .findById(id)
        .flatMap(order -> {
            return tacoRepo.findAllById(order.getTacoIds())
                .map(taco -> {
                    order.addTaco(taco);
                    return order;
                }).last();
        });
    }
}

```

The new `findById()` method is a bit shorter than the `save()` method. But we still have a lot to unpack in this small method.

The first thing to do is fetch the `TacoOrder` by calling the `findById()` method on the `OrderRepository`. This returns a `Mono<TacoOrder>` that is then flat-mapped to transform it from a `TacoOrder` that has only Taco IDs into a `TacoOrder` that includes complete Taco objects.

The lambda given to the `flatMap()` method makes a call to the `TacoRepository` `findAllById()` method to fetch all Taco objects referenced in the `tacoIds` property at once. This results in a `Flux<Taco>` that is cycled over via `map()`, adding each Taco to the parent `TacoOrder`, much like we did in the `save()` method after saving all Taco objects with `saveAll()`.

Again, the `map()` operation is used more as a means of iterating over the Taco objects rather than as a transformation. But the lambda given to `map()` returns the parent `TacoOrder` each time so that we end up with a `Flux<TacoOrder>` instead of a `Flux<Taco>`. The call to `last()` takes the last entry in that `Flux` and returns a `Mono<TacoOrder>`, which is what we return from the `findById()` method.

The code in the `save()` and `findById()` methods may be a little confusing if you're not already in a reactive mind-set. Reactive programming requires a different mindset and can be confusing at first, but you'll come to recognize it as quite elegant as your reactive programming skills get stronger.

As with any code—but especially code that may appear confusing like that in `TacoOrderAggregateService`—it's a good idea to write tests to ensure that it works as expected. The test will also serve as an example of how the `TacoOrderAggregateService` can be used. The following code listing shows a test for `TacoOrderAggregateService`.

Listing 13.11 Testing the `TacoOrderAggregateService`

```

package tacos.web.api;

import static org.assertj.core.api.Assertions.assertThat;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.r2dbc.DataR2dbcTest;
import org.springframework.test.annotation.DirtiesContext;

```

```

import reactor.test.StepVerifier;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.data.OrderRepository;
import tacos.data.TacoRepository;

@DataR2dbcTest
@DirtiesContext
public class TacoOrderAggregateServiceTests {

    @Autowired
    TacoRepository tacoRepo;

    @Autowired
    OrderRepository orderRepo;

    TacoOrderAggregateService service;

    @BeforeEach
    public void setup() {
        this.service = new TacoOrderAggregateService(tacoRepo, orderRepo);
    }

    @Test
    public void shouldSaveAndFetchOrders() {
        TacoOrder newOrder = new TacoOrder();
        newOrder.setDeliveryName("Test Customer");
        newOrder.setDeliveryStreet("1234 North Street");
        newOrder.setDeliveryCity("Notrees");
        newOrder.setDeliveryState("TX");
        newOrder.setDeliveryZip("79759");
        newOrder.setCcNumber("4111111111111111");
        newOrder.setCcExpiration("12/24");
        newOrder.setCcCVV("123");

        newOrder.addTaco(new Taco("Test Taco One"));
        newOrder.addTaco(new Taco("Test Taco Two"));

        StepVerifier.create(service.save(newOrder))
            .assertNext(this::assertOrder)
            .verifyComplete();

        StepVerifier.create(service.findById(1L))
            .assertNext(this::assertOrder)
            .verifyComplete();
    }

    private void assertOrder(TacoOrder savedOrder) {
        assertThat(savedOrder.getId()).isEqualTo(1L);
        assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
        assertThat(savedOrder.getDeliveryName()).isEqualTo("Test Customer");
        assertThat(savedOrder.getDeliveryStreet()).isEqualTo("1234 North Street");
        assertThat(savedOrder.getDeliveryCity()).isEqualTo("Notrees");
        assertThat(savedOrder.getDeliveryState()).isEqualTo("TX");
        assertThat(savedOrder.getDeliveryZip()).isEqualTo("79759");
    }
}

```

```

        assertThat(savedOrder.getCcNumber()).isEqualTo("4111111111111111");
        assertThat(savedOrder.getCcExpiration()).isEqualTo("12/24");
        assertThat(savedOrder.getCcCVV()).isEqualTo("123");
        assertThat(savedOrder.getTacoIds()).hasSize(2);
        assertThat(savedOrder.getTacos().get(0).getId()).isEqualTo(1L);
        assertThat(savedOrder.getTacos().get(0).getName())
            .isEqualTo("Test Taco One");
        assertThat(savedOrder.getTacos().get(1).getId()).isEqualTo(2L);
        assertThat(savedOrder.getTacos().get(1).getName())
            .isEqualTo("Test Taco Two");
    }
}

```

Listing 13.11 contains a lot of lines, but much of it is asserting the contents of a `TacoOrder` in the `assertOrder()` method. We'll focus on the other parts as we review this test.

The test class is annotated with `@DataR2dbcTest` to have Spring create an application context with all of our repositories as beans. `@DataR2dbcTest` seeks out a configuration class annotated with `@SpringBootTestConfiguration` to define the Spring application context. In a single-module project, the bootstrap class annotated with `@SpringBootTestConfiguration` (which itself is annotated with `@SpringBootTestConfiguration`) serves this purpose. But in our multimodule project, this test class isn't in the same project as the bootstrap class, so we'll need a simple configuration class like this one:

```

package tacos;

import org.springframework.boot.SpringBootTestConfiguration;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;

@SpringBootTestConfiguration
@EnableAutoConfiguration
public class TestConfig {

}

```

Not only does this satisfy the need for a `@SpringBootTestConfiguration`-annotated class, but it also enables autoconfiguration, ensuring that (among other things) the repository implementations will be created.

On its own, `TacoOrderAggregateServiceTests` should pass fine. But in an IDE that may share JVMs and Spring application contexts between test runs, running this test alongside other persistence tests may result in conflicting data being written to the in-memory H2 database. The `@DirtiesContext` annotation is used here to ensure that the Spring application context is reset between test runs, resulting in a new and empty H2 database on each run.

The `setup()` method creates an instance of `TacoOrderAggregateService` using the `TacoRepository` and `OrderRepository` objects injected into the test class. The `TacoOrderAggregateService` is assigned to an instance variable so that the test method(s) can use it.

Now we're finally ready to test our aggregation service. The first several lines of `shouldSaveAndFetchOrders()` builds up a `TacoOrder` object and populates it with a couple of test `Taco` objects. Then the `TacoOrder` is saved via the `save()` method from `TacoOrderAggregateService`, which returns a `Mono<TacoOrder>` representing the saved order. Using `StepVerifier`, we assert that the `TacoOrder` in the returned `Mono` matches our expectations, including that it contains the child `Taco` objects.

Next, we call the service's `findById()` method, which also returns a `Mono<TacoOrder>`. As with the call to `save()`, a `StepVerifier` is used to step through each `TacoOrder` in the returned `Mono` (there should be only one) and asserts that it meets our expectations.

In both `StepVerifier` situations, a call to `verifyComplete()` ensures that there are no more objects in the `Mono` and that the `Mono` is complete.

It's worth noting that although we could apply a similar aggregation operation to ensure that `Taco` objects always contain fully defined `Ingredient` objects, we choose not to, given that `Ingredient` is its own aggregate root, likely being referenced by multiple `Taco` objects. Therefore, every `Taco` will carry only a `Set<Long>` to reference `Ingredient` IDs, which can then be looked up separately via `IngredientRepository`.

Although it may require a bit more work to aggregate entities, Spring Data R2DBC provides a way of working with relational data in a reactive way. But it's not the only reactive persistence option provided by Spring. Let's have a look at how to work with MongoDB using reactive Spring Data repositories.

13.2 *Persisting document data reactively with MongoDB*

In chapter 4, we used Spring Data MongoDB to define document-based persistence against a MongoDB document database. In this section, we're going to revisit MongoDB persistence using Spring Data's reactive support for MongoDB.

To get started, you'll need to create a project with the Spring Data Reactive MongoDB starter. That is, in fact, the name of the check box to select when creating the project with the Initializr. Or you can add it manually to your Maven build with the following dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb-reactive</artifactId>
</dependency>
```

In chapter 4, we also leaned on the Flapdoodle embedded MongoDB database for testing. Unfortunately, Flapdoodle doesn't behave quite as well when fronted with reactive repositories. When it comes to running the tests, you'll need to have an actual Mongo database running and listening on port 27017.

Now we're ready to start writing code for reactive MongoDB persistence. We'll start with the document types that make up our domain.

13.2.1 Defining domain document types

As before, we'll need to create the classes that define our application's domain. As we do, we'll need to annotate them with Spring Data MongoDB's `@Document` annotation, just as we did in chapter 4, to indicate that they are documents to be stored in MongoDB. Let's start with the `Ingredient` class, shown here.

Listing 13.12 An `Ingredient` class annotated for Mongo persistence

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

A keen eye will notice that this `Ingredient` class is identical to the one we created in chapter 4. In fact, MongoDB `@Document` classes are the same whether being persisted through a reactive or nonreactive repository. That means that the `Taco` and `TacoOrder` classes are going to be the same as the ones we created in chapter 4. But for the sake of completeness—and so that you won't need to turn back to chapter 4—we'll repeat them here.

A similarly annotated `Taco` class is shown next.

Listing 13.13 A `Taco` class annotated for Mongo persistence

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
```

```

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.rest.core.annotation.RestResource;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Document
public class Taco {

    @Id
    private String id;

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}

```

Notice that, unlike `Ingredient`, the `Taco` class isn't annotated with `@Document`. That's because it isn't saved as a document in itself and is instead saved as part of the `TacoOrder` aggregate root. On the other hand, because `TacoOrder` is an aggregate root, it is annotated with `@Document` as shown in the next code.

Listing 13.14 A `TacoOrder` class annotated for Mongo persistence

```

package tacos;

import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document

```

```

public class TacoOrder implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    private String id;
    private Date placedAt = new Date();

    private User user;

    private String deliveryName;

    private String deliveryStreet;

    private String deliveryCity;

    private String deliveryState;

    private String deliveryZip;

    private String ccNumber;

    private String ccExpiration;

    private String ccCVV;

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

Again, the domain document classes are no different for reactive MongoDB repositories than they would be for nonreactive repositories. As you'll see next, reactive MongoDB repositories themselves differ very slightly from their nonreactive counterparts.

13.2.2 *Defining reactive MongoDB repositories*

Now we'll need to define two repositories, one for the TacoOrder aggregate root and another for Ingredient. We won't need a repository for Taco because it is a child of the TacoOrder root.

The IngredientRepository interface, shown here, should be familiar to you by now:

```

package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import org.springframework.web.bind.annotation.CrossOrigin;

import tacos.Ingredient;
@CrossOrigin(origins="http://localhost:8080")

```

```
public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, String> {

}
```

This `IngredientRepository` interface is only slightly different from the one we defined in chapter 4 in that it extends `ReactiveCrudRepository` instead of `CrudRepository`. And it differs from the one we created for Spring Data R2DBC persistence only in that it doesn't include the `findBySlug()` method.

Likewise, `OrderRepository` is all but identical to the same MongoDB repository we created in chapter 4, shown next:

```
package tacos.data;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, String> {

    Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
        User user, Pageable pageable);

}
```

Ultimately, the only difference between reactive and nonreactive MongoDB repositories is whether they extend `ReactiveCrudRepository` or `CrudRepository`. In choosing to extend `ReactiveCrudRepository`, however, clients of these repositories must be prepared to deal with reactive types like `Flux` and `Mono`. That becomes apparent as we write tests for the reactive repositories, which is what we'll do next.

13.2.3 Testing reactive MongoDB repositories

The key to writing tests for MongoDB repositories is to annotate the test class with `@DataMongoTest`. This annotation performs a function similar to the `@DataR2dbcTest` annotation that we used earlier in this chapter. It ensures that a Spring application context is created with the generated repositories available as beans to be injected into the test. From there, the test can use those injected repositories to set up test data and perform other operations against the database.

For example, consider `IngredientRepositoryTest` in the next listing, which tests `IngredientRepository`, asserting that `Ingredient` objects can be written to and read from the database.


```

        StepVerifier.create(ingredientRepo.findById("FLTO"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLTO", "Flour Tortilla",
                    Type.WRAP));
            });
    }
}

```

This test is similar to, but still slightly different from, the R2DBC-based repository test we wrote earlier in this chapter. It starts by writing three `Ingredient` objects to the database. Then, it employs two `StepVerifier` instances to verify that `Ingredient` objects can be read through the repository, first as a collection of all `Ingredient` objects and then fetching a single `Ingredient` by its ID.

Also, just as with the R2DBC-based test from earlier, the `@DataMongoTest` annotation will seek out a `@SpringBootTestConfiguration`-annotated class for creating the application context. A test just like the one created earlier will work here, too.

What's unique here is that the first `StepVerifier` collects all of the `Ingredient` objects into an `ArrayList` and then asserts that the `ArrayList` contains each `Ingredient`. The `findAll()` method doesn't guarantee a consistent ordering of the resulting documents, which makes the use of `assertNext()` or `expectNext()` prone to fail. By collecting all resulting `Ingredient` objects into a list, we can assert that the list has all three objects, regardless of their order.

A test for `OrderRepository` looks quite similar, as shown here.

Listing 13.16 Testing the Mongo OrderRepository

```

package tacos.data;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.mongo.DataMongoTest;

import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Taco;
import tacos.TacoOrder;
import tacos.Ingredient.Type;

@DataMongoTest
public class OrderRepositoryTest {

    @Autowired
    OrderRepository orderRepo;

    @BeforeEach
    public void setup() {
        orderRepo.deleteAll().subscribe();
    }
}

```

```

@Test
public void shouldSaveAndFetchOrders() {
    TacoOrder order = createOrder();

    StepVerifier
        .create(orderRepo.save(order))
        .expectNext(order)
        .verifyComplete();

    StepVerifier
        .create(orderRepo.findById(order.getId()))
        .expectNext(order)
        .verifyComplete();

    StepVerifier
        .create(orderRepo.findAll())
        .expectNext(order)
        .verifyComplete();
}

private TacoOrder createOrder() {
    TacoOrder order = new TacoOrder();
    ...
    return order;
}
}

```

The first thing that the `shouldSaveAndFetchOrders()` method does is construct an order, complete with customer and payment information and a couple of tacos. (For brevity's sake, I've left out the details of the `createOrder()` method.) It then uses a `StepVerifier` to save the `TacoOrder` object and assert that the `save()` method returns the saved `TacoOrder`. It then attempts to fetch the order by its ID and asserts that it receives the full `TacoOrder`. Finally, it fetches all `TacoOrder` objects—there should be only one—and asserts it is the expected `TacoOrder`.

As mentioned earlier, you'll need a MongoDB server available and listening on port 27017 to run this test. The Flapdoodle embedded MongoDB doesn't work well with reactive repositories. If you have Docker installed on your machine, you can easily start a MongoDB server exposed on port 27017 like this:

```
$ docker run -p27017:27017 mongo
```

Other ways to get a MongoDB setup are possible. Consult the documentation at <https://www.mongodb.com/> for more details.

Now that we've seen how to create reactive repositories for R2BDC and MongoDB, let's have a look at one more Spring Data option for reactive persistence: Cassandra.

13.3 Reactively persisting data in Cassandra

To get started with reactive persistence against a Cassandra database, you'll need to add the following starter dependency to your project build. This dependency is in lieu of any Mongo or R2DBC dependencies we've used earlier.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra-reactive</artifactId>
</dependency>
```

Then, you'll need to declare some details about the Cassandra keyspace and how the schema should be managed. In your `application.yml` file, add the following lines:

```
spring:
  data:
    rest:
      base-path: /data-api
    cassandra:
      keyspace-name: tacocloud
      schema-action: recreate
      local-datacenter: datacenter1
```

This is the same YAML configuration we used in chapter 4 when working with nonreactive Cassandra repositories. The key thing to take note of is the `keyspace-name`. It is important that you create a keyspace with that name in your Cassandra cluster.

You'll also need to have a Cassandra cluster running on your local machine listening on port 9042. The easiest way to do that is with Docker, as follows:

```
$ docker network create cassandra-net
$ docker run --name my-cassandra --network cassandra-net \
  -p 9042:9042 -d cassandra:latest
```

If your Cassandra cluster is on another machine or port, you'll need to specify the contact points and port in `application.yml`, as shown in chapter 4. To create the keyspace, run the CQL shell and use the `create keyspace` command like this:

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
cqlsh> create keyspace tacocloud
WITH replication = {'class': 'SimpleStrategy', 'replication_factor' : 1};
```

Now that you have a Cassandra cluster, a new `tacocloud` keyspace, and the Spring Data Cassandra Reactive starter in your project, you're ready to start defining the domain classes.

13.3.1 Defining domain classes for Cassandra persistence

As was the case when persisting with Mongo, the choice of reactive versus nonreactive Cassandra persistence makes absolutely no difference in how you define your domain classes. The domain classes for Ingredient, Taco, and TacoOrder we'll use are identical to the ones we created in chapter 4. A Cassandra-annotated Ingredient class is shown here.

Listing 13.17 Annotating Ingredient for Cassandra persistence

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

    @PrimaryKey
    private String id;
    private String name;
    private Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

As for the Taco class, it is defined with similar Cassandra persistence annotations in the next code listing.

Listing 13.18 Annotating Taco for Cassandra persistence

```
package tacos;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
```

```

import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;
import org.springframework.data.rest.core.annotation.RestResource;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@RestResource(rel = "tacos", path = "tacos")
@Table("tacos")
public class Taco {

    @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)
    private UUID id = Uuids.timeBased();

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,
                      ordering=Ordering.DESCENDING)
    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @Column("ingredients")
    private List<IngredientUDT> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(new IngredientUDT(ingredient.getName(),
        ingredient.getType()));
    }
}

```

Because Taco refers to Ingredient objects via a user-defined type, you'll also need the IngredientUDT class, as shown next.

Listing 13.19 An Ingredient user-defined type for Cassandra persistence

```

package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")

```

```
public class IngredientUDT {  
    private String name;  
    private Ingredient.Type type;  
}
```

The final of our three domain classes, `TacoOrder` is annotated for Cassandra persistence as shown in the following listing.

Listing 13.20 Annotating `TacoOrder` for Cassandra persistence

```
package tacos;  
  
import java.io.Serializable;  
import java.util.ArrayList;  
import java.util.Date;  
import java.util.List;  
import java.util.UUID;  
  
import org.springframework.data.cassandra.core.mapping.Column;  
import org.springframework.data.cassandra.core.mapping.PrimaryKey;  
import org.springframework.data.cassandra.core.mapping.Table;  
  
import com.datastax.oss.driver.api.core.uuid.Uuids;  
  
import lombok.Data;  
  
@Data  
@Table("tacoorders")  
public class TacoOrder implements Serializable {  
    private static final long serialVersionUID = 1L;  
  
    @PrimaryKey  
    private UUID id = Uuids.timeBased();  
    private Date placedAt = new Date();  
  
    @Column("user")  
    private UserUDT user;  
  
    private String deliveryName;  
  
    private String deliveryStreet;  
  
    private String deliveryCity;  
  
    private String deliveryState;  
  
    private String deliveryZip;  
  
    private String ccNumber;  
  
    private String ccExpiration;  
  
    private String ccCVV;
```

```

@Column("tacos")
private List<TacoUDT> tacos = new ArrayList<>();

public void addTaco(Taco taco) {
    this.addTaco(new TacoUDT(taco.getName(), taco.getIngredients()));
}

public void addTaco(TacoUDT tacoUDT) {
    this.tacos.add(tacoUDT);
}
}

```

And, just like how Taco refers to Ingredient via a user-defined type, TacoOrder refers to Taco via the TacoUDT class, which is shown next.

Listing 13.21 An Taco user-defined type for Cassandra persistence

```

package tacos;

import java.util.List;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;
import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

    private final String name;
    private final List<IngredientUDT> ingredients;

}

```

It bears repeating that these are identical to their nonreactive counterparts. I've only repeated them here so that you don't have to flip back 11 chapters to remember what they look like.

Now let's define the repositories that persist these objects.

13.3.2 Creating reactive Cassandra repositories

By now you may already be expecting the reactive Cassandra repositories to look a lot like the equivalent nonreactive repositories. If so, then great! You're catching on that Spring Data, wherever possible, attempts to maintain a similar programming model regardless of whether or not repositories are reactive.

You may have already guessed that the only key difference that makes the repositories reactive is that the interfaces extend `ReactiveCrudRepository`, as shown here in the `IngredientRepository` interface:


```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends ReactiveCrudRepository<Ingredient, String> {

}
```

Naturally, the same holds true for `OrderRepository`, as shown next:

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.domain.Pageable;
import org.springframework.data.repository.reactive.ReactiveCrudRepository;

import reactor.core.publisher.Flux;
import tacos.TacoOrder;
import tacos.User;

public interface OrderRepository
    extends ReactiveCrudRepository<TacoOrder, UUID> {

    Flux<TacoOrder> findByUserOrderByPlacedAtDesc(
        User user, Pageable pageable);

}
```

In fact, not only are these repositories reminiscent of their nonreactive counterparts, they also do not differ greatly from the MongoDB repositories we wrote earlier in this chapter. Aside from Cassandra using `UUID` as an ID type instead of `String` for `TacoOrder`, they are virtually identical. This once again demonstrates the consistency employed (where possible) across Spring Data projects.

Let's wrap up our look at writing reactive Cassandra repositories by writing a couple of tests to verify that they work.

13.3.3 *Testing reactive Cassandra repositories*

At this point, it may not come as a surprise that testing reactive Cassandra repositories is quite similar to how you test reactive MongoDB repositories. For example, take a look at `IngredientRepositoryTest` in the next listing, and see if you can spot how it differs from listing 13.15.

Listing 13.22 Testing the Cassandra `IngredientRepository`

```
package tacos.data;

import static org.assertj.core.api.Assertions.assertThat;
```

```

import java.util.ArrayList;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.data.cassandra
    .DataCassandraTest;

import reactor.core.publisher.Flux;
import reactor.test.StepVerifier;
import tacos.Ingredient;
import tacos.Ingredient.Type;

@DataCassandraTest
public class IngredientRepositoryTest {

    @Autowired
    IngredientRepository ingredientRepo;

    @BeforeEach
    public void setup() {
        Flux<Ingredient> deleteAndInsert = ingredientRepo.deleteAll()
            .thenMany(ingredientRepo.saveAll(
                Flux.just(
                    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE)
                )))
            .step();

        StepVerifier.create(deleteAndInsert)
            .expectNextCount(3)
            .verifyComplete();
    }

    @Test
    public void shouldSaveAndFetchIngredients() {

        StepVerifier.create(ingredientRepo.findAll())
            .recordWith(ArrayList::new)
            .thenConsumeWhile(x -> true)
            .consumeRecordedWith(ingredients -> {
                assertThat(ingredients).hasSize(3);
                assertThat(ingredients).contains(
                    new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
                assertThat(ingredients).contains(
                    new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
                assertThat(ingredients).contains(
                    new Ingredient("CHED", "Cheddar Cheese", Type.CHEESE));
            })
            .verifyComplete();

        StepVerifier.create(ingredientRepo.findById("FLTO"))
            .assertNext(ingredient -> {
                ingredient.equals(new Ingredient("FLTO", "Flour Tortilla",
                    Type.WRAP));
            })
            .verifyComplete();
    }
}

```

```

        });
    }
}

```

Did you see it? Where the MongoDB version was annotated with `@DataMongoTest`, this new Cassandra version is annotated with `@DataCassandraTest`. That's it! Otherwise, the tests are identical.

The same is true for `OrderRepositoryTest`. Replace `@DataMongoTest` with `@DataCassandraTest`, and everything else is the same, as shown here:

```

@DataCassandraTest
public class OrderRepositoryTest {
    ...
}

```

Once again, consistency between various Spring Data projects extends even into how the tests are written. This makes it easy to switch between projects that persist to different kinds of databases without having to think much differently about how they are developed.

Summary

- Spring Data supports reactive persistence for a variety of database types, including relational databases (with R2DBC), MongoDB, and Cassandra.
- Spring Data R2DBC offers a reactive option for relational persistence but doesn't yet directly support relationships in domain classes.
- For lack of direct relationship support, Spring Data R2DBC repositories require a different approach to domain and database table design.
- Spring Data MongoDB and Spring Data Cassandra offer a near-identical programming model for writing reactive repositories for MongoDB and Cassandra databases.
- Using Spring Data test annotations along with `StepVerifier`, you can test automatically created reactive repositories from the Spring application context.

14

Working with RSocket

This chapter covers

- Reactive network communication with RSocket
- Working with each of RSocket's four communication models
- Transporting RSocket over WebSocket

There was a time, before telephones and modern electronics, when the best way to communicate with friends and family that live far away involved writing a letter and dropping it in the mail. It wasn't a quick form of communication, taking several days or even weeks before you'd receive a response, but it was effective and truly the only option available.

Thanks to Alexander Graham Bell, the telephone offered a new way to talk with distant friends and family, giving near-real-time, synchronous communication. The telephone has evolved quite a bit since Mr. Bell's first invention, but it's still a popular means of keeping in touch, making letter-writing nearly a lost art.

When it comes to communication between applications, the request-response model offered by HTTP and REST services is quite common, but it has limitations. Much like letter-writing, request-response involves sending a message and then waiting for a response. It doesn't easily allow for asynchronous communication in

which a server might respond with a stream of responses or allow for an open bidirectional channel on which a client and server can repeatedly send data to each other.

In this chapter, we're going to look at RSocket, a relatively new protocol for inter-application communication that allows for more than simple request-response communication. And because it's reactive in nature, it can be far more efficient than blocking HTTP requests.

Along the way, we'll see how to develop RSocket communication in Spring. But first, let's take a high-level look at RSocket to see what makes it different from HTTP-based communication.

14.1 Introducing RSocket

RSocket (<https://rsocket.io/>) is a binary application protocol that is asynchronous and based on Reactive Streams. Put another way, RSocket offers asynchronous communication between applications that supports a reactive model consistent with reactive types like Flux and Mono that we learned about in chapter 12.

As an alternative to HTTP-based communication, it is more flexible, providing four distinct communication models: request-response, request-stream, fire-and-forget, and channel.

Request-response is the most familiar communication model from RSocket, mimicking how typical HTTP communication works. In the request-response model, a client issues a single request to the server, and the server responds with a single response. This is illustrated in figure 14.1, using Reactor's Mono type to define the request and response.

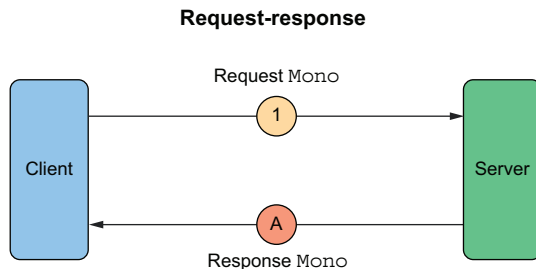


Figure 14.1 RSocket's request-response communication model

Although the request-response model may appear to be equivalent to the communication model offered by HTTP, it's important to understand that RSocket is fundamentally nonblocking and based on reactive types. Although the client will still wait for a reply from the server, under the covers everything is nonblocking and reactive, making more efficient use of threads.

The *request-stream* communication model is similar to request-response, except that after the client has sent a single request to the server, the server responds with a stream of zero-to-many values in a stream. Figure 14.2 illustrates the request-stream model using Mono for the request and Flux for the response.

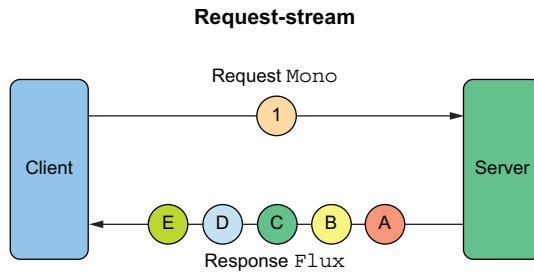


Figure 14.2 RSocket's request-stream communication model

In some cases, the client may need to send data to the server but doesn't need a response. RSocket provides the *fire-and-forget* model for those situations, as illustrated in figure 14.3.

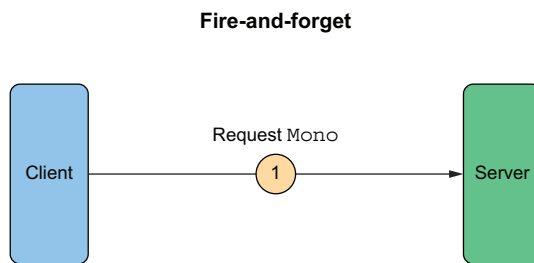


Figure 14.3 RSocket's fire-and-forget communication model

In the fire-and-forget model, a client sends a request to the server, but the server doesn't send a response back.

Finally, the most flexible of RSocket's communication models is the *channel* model. In the channel model, the client opens a bidirectional channel with the server, and each can send data to the other at any time. Figure 14.4 illustrates the channel communication style.

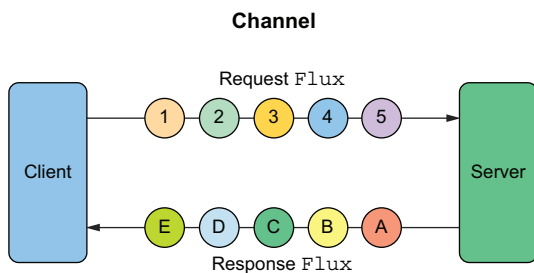


Figure 14.4 RSocket's channel communication model

RSocket is supported on a variety of languages and platforms, including Java, JavaScript, Kotlin, .NET, Go, and C++. ¹ Recent versions of Spring offer first-class support for RSocket, making it easy to create servers and clients using familiar Spring idioms.

¹ This is just a short list of languages that are listed on the RSocket website, but there may be community-led implementations of RSocket for other languages.

Let's dive in and see how to create RSocket servers and clients that work with each of the four communication models.

14.2 *Creating a simple RSocket server and client*

Spring offers incredible support for messaging with RSocket, including all four communication models. To get started with RSocket, you'll need to add the Spring Boot RSocket starter to your project's build. In a Maven POM file, the RSocket starter dependency looks like this the following.

Listing 14.1 Spring Boot's RSocket starter dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-rsocket</artifactId>
</dependency>
```

This same dependency is needed for both the server and client applications involved in RSocket communication.

NOTE When choosing dependencies from the Spring Initializr, you might see a similarly named WebSocket dependency. Although RSocket and WebSocket have similar names and although you can use WebSocket as a transport for RSocket (and we'll cover that later in this chapter), you do not need to select the WebSocket dependency when working with RSocket.

Next, you'll need to decide which communication model is best for your application. There's no clear answer that fits every situation, so you'll want to weigh the choice against the desired communication behavior of your application. However, as you'll see in the next several examples, the development model isn't much different for each of the communication models, so it'll be easy to switch if you choose wrong.

Let's see how to create an RSocket server and client in Spring using each of the communication models. Because each of RSocket's communication models is different and is best suited for specific use-case scenarios, we'll set the Taco Cloud application aside for now and see how to apply RSocket on different problem domains. We'll start by seeing how to apply the request-response communication model.

14.2.1 *Working with request-response*

Creating an RSocket server in Spring is as simple as creating a controller class, much the same as you would for a web application or REST service. The following controller is an example of an RSocket service that handles greetings from the client and responds with another greeting.

Listing 14.2 A simple RSocket request-response server

```
package rsocket;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
```

```
import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;

@Controller
@Slf4j
public class GreetingController {

    @MessageMapping("greeting")
    public Mono<String> handleGreeting(Mono<String> greetingMono) {
        return greetingMono
            .doOnNext(greeting ->
                log.info("Received a greeting: {}", greeting))
            .map(greeting -> "Hello back to you!");
    }
}
```

As you can see, the key difference between a web controller and an RSocket controller is that instead of handling HTTP requests for a given path (using `@GetMapping` or `@PostMapping`), an RSocket controller handles incoming messages on a given route with the `@MessageMapping` annotation. In this example, the `handleGreeting()` method is invoked when a request is sent from the client to the route named "greeting".

The `handleGreeting()` method receives the message payload from the client in a `Mono<String>` parameter. In this case, the greeting is simple enough that a `String` is sufficient, but the incoming payload could be a more complex type, if needed. Upon receiving the `Mono<String>`, the method simply logs the fact that it received the greeting and then uses the `map()` function on the `Mono` to create a new `Mono<String>` to carry the response that is returned to the client.

Although RSocket controllers aren't handling HTTP requests for a path, the route name can be made to have a pathlike appearance, including variable placeholders that can be passed into the handler method. For example, consider the following twist on the `handleGreeting()` method:

```
@MessageMapping("greeting/{name}")
public Mono<String> handleGreeting(
    @DestinationVariable("name") String name,
    Mono<String> greetingMono) {

    return greetingMono
        .doOnNext(greeting ->
            log.info("Received a greeting from {} : {}", name, greeting))
        .map(greeting -> "Hello to you, too, " + name);
}
```

In this case, the route specified in `@MessageMapping` contains a placeholder variable named "name". It is denoted by curly braces, the same way as path variables in a Spring MVC controller. Likewise, the method accepts a `String` parameter annotated with `@DestinationVariable` that references the placeholder variable. Just like Spring

MVC's `@PathVariable` annotation, `@DestinationVariable` is used to extract the value specified in the route's placeholder and pass it into the handler method. Once inside this new version of `handleGreeting()`, the name specified in the route will be used to return a more personalized greeting to the client.

There's one more thing you must remember to do when creating an RSocket server: specify the port to listen on. By default, RSocket services are TCP-based and are their own server listening on a specific port. The `spring.rsocket.server.port` configuration property sets the port for the RSocket server, as shown here:

```
spring:
  rsocket:
    server:
      port: 7000
```

The `spring.rsocket.server.port` property serves two purposes: enabling a server and specifying which port the server should listen on. If it is not set, then Spring will assume that your application will be acting as a client only, and no server port will be listening. In this case, we're starting a server, so setting the `spring.rsocket.server.port` property as shown in the previous code will start a server listening on port 7000.

Now let's turn our attention to the RSocket client. In Spring, RSocket clients are implemented using an `RSocketRequester`. Spring Boot autoconfiguration for RSocket will automatically create a bean of type `RSocketRequester.Builder` in the Spring application context. You can inject that builder bean into any other bean you need to create an instance of `RSocketRequester`.

For example, here's the start of an `ApplicationRunner` bean that is injected with an `RSocketRequester.Builder`:

```
package rsocket;
import org.springframework.boot.ApplicationRunner;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.messaging.rsocket.RSocketRequester;

@Configuration
@Slf4j
public class RSocketClientConfiguration {

    @Bean
    public ApplicationRunner sender(RSocketRequester.Builder requesterBuilder)
    {
        return args -> {
            RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

            // ... send messages with RSocketRequester ...

        };
    }
}
```

In this case, the builder is used to create an `RSocketRequester` that listens on localhost, port 7000. The resulting `RSocketRequester` can then be used to send messages to the server.

In a request-response model, the request will need to (at least) specify the route and the data payload. As you'll recall, our server's controller is handling requests for the route named "greeting" and expects a `String` input. It also returns a `String` output. The following complete listing of client code shows how to send a greeting to the server and handle the response.

Listing 14.3 Sending a request from a client

```
RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

// ... send messages with RSocketRequester ...
tcp
    .route("greeting")
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: {}", response));
```

This sends a greeting of "Hello RSocket!" to the server on the "greeting" route. Notice that it also expects a `Mono<String>` in return, as specified in the call to `retrieveMono()`. The `subscribe()` method subscribes to the returned `Mono` and handles its payload by logging the value.

Now let's say you want to send a greeting to the other route that accepts a variable value in its route. The client-side code works pretty much the same, except that you include the variable placeholder in the value given to `route()` along with the value it should contain as follows:

```
String who = "Craig";
tcp
    .route("greeting/{name}", who)
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: {}", response));
```

Here, the message will be sent to the route named "greeting/Craig", which will be handled by the controller handler method whose `@RequestMapping` specified the route "greeting/{name}". Although you could also hardcode the name in the route or use `String` concatenation to create the route name, using a placeholder in the client makes it really easy to drop in a value without the messiness of `String` concatenation.

Although the request-response model is probably the easiest of RSocket's communication models to wrap your head around, it's just the beginning. Let's see how to handle requests that could potentially return several responses with the request-stream model.

14.2.2 Handling request-stream messaging

Not all interactions feature a single request and a single response. In a stock quote scenario, for example, it may be useful to request a stream of stock quotes for a given stock symbol. In a request-response model, the client would need to repeatedly poll for the current stock price. But in a request-stream model, the client need ask for the stock price only once and then subscribe to a stream of periodic updates.

To illustrate the request-stream model, let's implement the server and client for the stock quote scenario. First, we'll need to define an object that can carry the stock quote information. The `StockQuote` class in the next listing will serve this purpose.

Listing 14.4 A model class representing a stock quote

```
package rsocket;
import java.math.BigDecimal;
import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class StockQuote {

    private String symbol;
    private BigDecimal price;
    private Instant timestamp;
}
```

As you can see, a `StockQuote` carries the stock symbol, the price, and a timestamp that the price was valid. For brevity's sake, we're using Lombok to help with constructors and accessor methods.

Now let's write a controller to handle requests for stock quotes. You'll find that the `StockQuoteController` in the next snippet is quite similar to the `GreetingController` from the previous section.

Listing 14.5 An RSocket controller to stream stock quotes

```
package rsocket;
import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;

import org.springframework.messaging.handler.annotation.DestinationVariable;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import reactor.core.publisher.Flux;
```

```

@Controller
public class StockQuoteController {

    @MessageMapping("stock/{symbol}")
    public Flux<StockQuote> getStockPrice(
        @DestinationVariable("symbol") String symbol) {
        return Flux
            .interval(Duration.ofSeconds(1))
            .map(i -> {
                BigDecimal price = BigDecimal.valueOf(Math.random() * 10);
                return new StockQuote(symbol, price, Instant.now());
            });
    }
}

```

Here, the `getStockPrice()` method handles incoming requests on the `"stock/{symbol}"` route, accepting the stock symbol from the route with the `@DestinationVariable` annotation. For simplicity's sake, rather than look up actual stock prices, the price is calculated as a random value (which may or may not accurately model the volatility of some actual stocks).

What's most notable about `getStockPrice()`, however, is that it returns a `Flux<StockQuote>` instead of a `Mono<StockQuote>`. This is a clue to Spring that this handler method supports the request-stream model. Internally, the `Flux` is created initially as an interval that fires every one second, but that `Flux` is mapped to another `Flux` that produces the random `StockQuote`. Put simply, a single request handled by the `getStockPrice()` method returns multiple values, once every second.

A client of a request-stream service is similar to one for a request-response service. The only key difference is that instead of calling `retrieveMono()` on the requester, it should call `retrieveFlux()`. The client of the stock quote service might look like this:

```

String stockSymbol = "XYZ";

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
    .route("stock/{symbol}", stockSymbol)
    .retrieveFlux(StockQuote.class)
    .doOnNext(stockQuote ->
        log.info(
            "Price of {} : {} (at {})",
            stockQuote.getSymbol(),
            stockQuote.getPrice(),
            stockQuote.getTimestamp())
    )
    .subscribe();

```

At this point, we've seen how to create RSocket servers and clients that handle single and multiple responses. But what if the server doesn't have a response to send or the client doesn't need a response? Let's see how to deal with the fire-and-forget communication model.

14.2.3 Sending fire-and-forget messages

Imagine that you're on a starship that has just come under attack from an enemy vessel. You sound a ship-wide "red alert" so that all hands are in battle mode. You don't need to wait for a response from the ship's computers affirming the alert status, nor do you have time to wait for and read any kind of response in this situation. You set the alert and then move on to more critical matters.

This is an example of fire-and-forget. Although you may not forget that you're at red alert, given the circumstances, it's more important that you deal with the battle crisis than it is for you to handle a response from setting the alert.

To simulate this scenario, we'll create an RSocket server that handles alert statuses but doesn't return anything. First, we'll need to define a class that defines the request payload, such as the `Alert` class in the following code listing.

Listing 14.6 A model class representing an alert

```
package rsocket;

import java.time.Instant;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class Alert {

    private Level level;
    private String orderedBy;
    private Instant orderedAt;

    public enum Level {
        YELLOW, ORANGE, RED, BLACK
    }
}
```

The `Alert` object captures the alert level, who ordered the alert, and a timestamp for when the alert was ordered (defined as an `Instant`). Again, we're using Lombok for constructors and accessor methods in the interest of keeping the listing short.

On the server side, the `AlertController` in the following listing will handle `Alert` messages.

Listing 14.7 An RSocket controller to handle alert updates

```
package rsocket;

import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;
import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Mono;
```

```

@Controller
@Slf4j
public class AlertController {

    @PostMapping("alert")
    public Mono<Void> setAlert(Mono<Alert> alertMono) {
        return alertMono
            .doOnNext(alert ->
                log.info("{} alert ordered by {} at {}",
                    alert.getLevel(),
                    alert.getOrderedBy(),
                    alert.getOrderedAt())
            )
            .thenEmpty(Mono.empty());
    }
}

```

The `setAlert()` method handles Alert messages on the "alert" route. To keep things simple (albeit useless in an actual battle situation), it logs only the alerts. But what's important is that it returns a `Mono<Void>`, indicating that there is no response, and, therefore, this handler method supports the fire-and-forget model.

In the client, the code isn't much different from the request-response or request-stream models, as shown here:

```

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);
tcp
    .route("alert")
    .data(new Alert(
        Alert.Level.RED, "Craig", Instant.now()))
    .send()
    .subscribe();
log.info("Alert sent");

```

Notice, however, that instead of calling `retrieveMono()` or `retrieveFlux()`, the client merely calls `send()` with no expectation of a response.

Now let's take a look at how to handle the channel communication model in which both the server and the client send multiple messages to each other.

14.2.4 Sending messages bidirectionally

In all of the communication models we've seen thus far, the client sends a single request, and the server responds with zero, one, or many responses. In the request-stream model, the server was able to stream back multiple responses to the client, but the client was still limited to sending only a single request. But why should the server have all of the fun? Why can't the client send multiple requests?

That's where the channel communication model comes in handy. In the channel communication model, the client can stream multiple requests to the server, which may also stream back multiple responses in a bidirectional conversation between both

sides. It's the most flexible of RSocket's communication models, although also the most complex.

To demonstrate how to work with RSocket channel communication in Spring, let's create a service that calculates gratuity on a bill, receiving a Flux of requests and responding with a Flux of responses. First, we'll need to define the model objects that represent the request and the response. The `GratuityIn` class, shown next, represents the request sent by the client and received by the server.

Listing 14.8 A model representing an inbound gratuity request

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratuityIn {

    private BigDecimal billTotal;
    private int percent;

}
```

`GratuityIn` carries two essential pieces of information required to calculate gratuity: the bill total and a percentage. The `GratuityOut` class shown in the next code snippet represents the response, echoing the values given in `GratuityIn`, along with a `gratuity` property containing the calculated gratuity amount.

Listing 14.9 A model representing an outbound gratuity response

```
package rsocket;

import java.math.BigDecimal;

import lombok.AllArgsConstructor;
import lombok.Data;

@Data
@AllArgsConstructor
public class GratuityOut {

    private BigDecimal billTotal;
    private int percent;
    private BigDecimal gratuity;

}
```

The `GratuityController` in the next code listing handles the gratuity request and looks a lot like the controllers we've written earlier in this chapter.

Listing 14.10 An RSocket controller that handles multiple messages on a channel

```

package rsocket;
import java.math.BigDecimal;
import org.springframework.messaging.handler.annotation.MessageMapping;
import org.springframework.stereotype.Controller;

import lombok.extern.slf4j.Slf4j;
import reactor.core.publisher.Flux;

@Controller
@Slf4j
public class GratuityController {

    @MessageMapping("gratuity")
    public Flux<GratuityOut> calculate(Flux<GratuityIn> gratuityInFlux) {
        return gratuityInFlux
            .doOnNext(in -> log.info("Calculating gratuity: {}", in))
            .map(in -> {
                double percentAsDecimal = in.getPercent() / 100.0;
                BigDecimal gratuity = in.getBillTotal()
                    .multiply(BigDecimal.valueOf(percentAsDecimal));
                return new GratuityOut(
                    in.getBillTotal(), in.getPercent(), gratuity);
            });
    }
}

```

There is, however, one significant difference between the previous example and the earlier ones: not only does this code return a Flux, but it also accepts a Flux as input. As with the request-stream model, the Flux returned enables the controller to stream multiple values to the client. But the Flux parameter is what differentiates the channel model from the request-stream model. The Flux parameter coming in allows the controller to handle a stream of requests from the client coming into the handler method.

The client side of the channel model differs from the client of the request-stream model only in that it sends a Flux<GratuityIn> to the server instead of a Mono<GratuityIn>, as shown here.

Listing 14.11 A client that sends and receives multiple messages over an open channel

```

RSocketRequester tcp = requesterBuilder.tcp("localhost", 7000);

Flux<GratuityIn> gratuityInFlux =
    Flux.fromArray(new GratuityIn[] {
        new GratuityIn(BigDecimal.valueOf(35.50), 18),
        new GratuityIn(BigDecimal.valueOf(10.00), 15),
        new GratuityIn(BigDecimal.valueOf(23.25), 20),
        new GratuityIn(BigDecimal.valueOf(52.75), 18),
    })

```



```

        new GratuuityIn(BigDecimal.valueOf(80.00), 15)
    })
    .delayElements(Duration.ofSeconds(1));

tcp
    .route("gratuuity")
    .data(gratuuityInFlux)
    .retrieveFlux(GratuuityOut.class)
    .subscribe(out ->
        log.info(out.getPercent() + "% gratuuity on "
            + out.getBillTotal() + " is "
            + out.getGratuuity()));

```

In this case, the `Flux<GratuuityIn>` is created statically using the `fromArray()` method, but it could be a `Flux` created from any source of data, perhaps retrieved from a reactive data repository.

You may have observed a pattern in how the reactive types accepted and returned by the server controller's handler methods determine the RSocket communication model supported. Table 14.1 summarizes the relationship between the server's input/output types and the RSocket communication models.

Table 14.1 The supported RSocket model is determined by the handler method's parameter and return types.

RSocket model	Handler parameter	Handler returns
Request-response	Mono	Mono
Request-stream	Mono	Flux
Fire-and-forget	Mono	Mono<Void>
Channel	Flux	Flux

You may wonder whether it's possible for a server to accept a `Flux` and return a `Mono`. In short, that's not an option. Although you may imagine handling multiple requests on an incoming `Flux` and responding with a `Mono<Void>` in a weird mashup of the channel and fire-and-forget models, there is no RSocket model that maps to that scenario. Therefore, it's not supported.

14.3 *Transporting RSocket over WebSocket*

By default, RSocket communication takes place over a TCP socket. But in some cases, TCP isn't an option. Consider the following two situations:

- The client is written in JavaScript and is running in a user's web browser.
- The client must cross a gateway or firewall boundary to get to the server, and the firewall doesn't allow communication over arbitrary ports.

Moreover, WebSocket itself lacks any support for routing, requiring that routing details be defined at the application level. By layering RSocket over WebSocket, WebSocket will benefit from RSocket's built-in routing support.

In these situations, RSocket can be transported over WebSocket. WebSocket communication takes place over HTTP, which is the primary means of communication in all web browsers and is usually allowed through firewalls.

To switch from the TCP transport to the WebSocket transport, you need to make only a few minor changes in the server and client. To start, because WebSocket is carried over HTTP, you need to be sure that the server-side application supports handling HTTP requests. Put simply, you need to add the following WebFlux starter dependency to the project build (if it's not already there):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

You also need to specify that you want to use the WebSocket transport in the server-side configuration by setting the `spring.rsocket.server.transport` property. Also, you need to set the HTTP path that the RSocket communication will take place on by setting `spring.rsocket.server.mapping-path`. The server's configuration will look like this in `application.yml`:

```
spring:
  rsocket:
    server:
      transport: websocket
      mapping-path: /rsocket
```

Unlike the TCP transport, which communicates over a specific port, the WebSocket transport works over a specific HTTP path. Thus, there is no need to set `spring.rsocket.server.port` as with RSocket over TCP.

That's all you'll need to do on the server side to enable WebSocket transport for RSocket. Everything else will work exactly the same as with TCP.

On the client side, only one small change is required. Rather than create a TCP-based requester, you want to create a WebSocket-based requester by calling the `websocket()` method on the `RSocketRequester.Builder` like so:

```
RSocketRequester requester = requesterBuilder.websocket(
    URI.create("ws://localhost:8080/rsocket"));

requester
    .route("greeting")
    .data("Hello RSocket!")
    .retrieveMono(String.class)
    .subscribe(response -> log.info("Got a response: {}", response));
```

And that's all there is to transporting RSocket over WebSocket!

Summary

- RSocket is an asynchronous binary protocol that offers four communication models: request-response, request-stream, fire-and-forget, and channel.
- Spring supports RSocket on the server through controllers and handler methods annotated with `@MessageHandler`.
- The `RSocketRequester` enables client-side communication with RSocket.
- In both cases, Spring's RSocket support works through Reactor's `Flux` and `Mono` reactive types for fully reactive communication.
- RSocket communication takes place over TCP by default but can also be transported over WebSocket to deal with firewall constraints and browser clients.

Part 4

Deployed Spring

In part 4, you'll ready an application for production and see how to deploy it. Chapter 15 introduces the Spring Boot Actuator, an extension to Spring Boot that exposes the internals of a running Spring application as REST endpoints and JMX MBeans. In chapter 16, you'll see how to use Spring Boot Admin to put a user-friendly browser-based administrative application on top of the Actuator. You'll also see how to register client applications with and secure the Admin Server. Chapter 17 discusses how to expose and consume Spring beans as JMX MBeans. Finally, in chapter 18, you'll see how to deploy your Spring application in a variety of production environments, including containerized Spring applications that can run in Kubernetes.

15

Working with Spring Boot Actuator

This chapter covers

- Enabling Actuator in Spring Boot projects
- Exploring Actuator endpoints
- Customizing Actuator
- Securing Actuator

Have you ever tried to guess what's inside a wrapped gift? You shake it, weigh it, and measure it. And you might even have a solid idea as to what's inside. But until you open it up, there's no way of knowing for sure.

A running application is kind of like a wrapped gift. You can poke at it and make reasonable guesses as to what's going on under the covers. But how can you know for sure? If only there were some way that you could peek inside a running application, see how it's behaving, check on its health, and maybe even trigger operations that influence how it runs!

In this chapter, we're going to explore Spring Boot Actuator. Actuator offers production-ready features such as monitoring and metrics to Spring Boot applications. Actuator's features are provided by way of several endpoints, which are made available over HTTP as well as through JMX MBeans. This chapter focuses primarily on HTTP endpoints, saving JMX endpoints for chapter 18.

15.1 Introducing Actuator

In a machine, an actuator is a component that's responsible for controlling and moving a mechanism. In a Spring Boot application, the Spring Boot Actuator plays that same role, enabling us to see inside of a running application and, to some degree, control how the application behaves.

Using endpoints exposed by Actuator, we can ask things about the internal state of a running Spring Boot application, such as the following:

- What configuration properties are available in the application environment?
- What are the logging levels of various packages in the application?
- How much memory is being consumed by the application?
- How many times has a given HTTP endpoint been requested?
- What is the health of the application and any external services it coordinates with?

To enable Actuator in a Spring Boot application, you simply need to add Actuator's starter dependency to your build. In any Spring Boot application Maven pom.xml file, the following <dependency> entry does the trick:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Once the Actuator starter is part of the project build, the application will be equipped with several out-of-the-box Actuator endpoints, including those described in table 15.1.

Table 15.1 Actuator endpoints for peeking inside and manipulating the state of a running Spring Boot application

HTTP method	Path	Description
GET	/auditevents	Produces a report of any audit events that have been fired
GET	/beans	Describes all the beans in the Spring application context
GET	/conditions	Produces a report of autoconfiguration conditions that either passed or failed, leading to the beans created in the application context
GET	/configprops	Describes all configuration properties along with the current values
GET, POST, DELETE	/env	Produces a report of all property sources and their properties available to the Spring application
GET	/env/{toMatch}	Describes the value of a single environment property
GET	/health	Returns the aggregate health of the application and (possibly) the health of external dependent applications
GET	/heapdump	Downloads a heap dump

Table 15.1 Actuator endpoints for peeking inside and manipulating the state of a running Spring Boot application (*continued*)

HTTP method	Path	Description
GET	/httptrace	Produces a trace of the most recent 100 requests
GET	/info	Returns any developer-defined information about the application
GET	/loggers	Produces a list of packages in the application along with their configured and effective logging levels
GET, POST	/loggers/{name}	Returns the configured and effective logging level of a given logger; the effective logging level can be set with a POST request
GET	/mappings	Produces a report of all HTTP mappings and their corresponding handler methods
GET	/metrics	Returns a list of all metrics categories
GET	/metrics/{name}	Returns a multidimensional set of values for a given metric
GET	/scheduledtasks	Lists all scheduled tasks
GET	/threaddump	Returns a report of all application threads

In addition to HTTP endpoints, all of the Actuator endpoints in table 15.1, with the lone exception of `/heapdump`, are also exposed as JMX MBeans. We'll look at the JMX side of Actuator in chapter 17.

15.1.1 Configuring Actuator's base path

By default, the paths for all the endpoints shown in table 15.1 are prefixed with `/actuator`. This means that, for example, if you wish to retrieve health information about your application from Actuator, then issuing a GET request for `/actuator/health` will return the information you need.

The Actuator prefix path can be changed by setting the `management.endpoint.web.base-path` property. For example, if you'd rather the prefix be `/management`, you would set the `management.endpoints.web.base-path` property like this:

```
management:
  endpoints:
    web:
      base-path: /management
```

With this property set as shown, you'd need to make a GET request for `/management/health` to obtain the application's health information.

Whether or not you decide to change the Actuator base path, all Actuator endpoints in this chapter will be referred to without the base path for the sake of brevity. For example, when the `/health` endpoint is mentioned, it is the `{base path}/health`

endpoint that is intended, or more precisely, the `/actuator/health` endpoint if the base path hasn't been changed.

15.1.2 *Enabling and disabling Actuator endpoints*

You may have noticed that only the `/health` endpoint is enabled by default. Most Actuator endpoints carry sensitive information and should be secured. You can use Spring Security to lock down Actuator, but because Actuator isn't secured on its own, most of the endpoints are disabled by default, requiring you to opt in for the endpoints you wish to expose.

Two configuration properties, `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude`, can be used to control which endpoints are exposed. Use `management.endpoints.web.exposure.include` to specify which endpoints you want to expose. For example, if you wish to expose only the `/health`, `/info`, `/beans`, and `/conditions` endpoints, you can specify that with the following configuration:

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,beans,conditions
```

The `management.endpoints.web.exposure.include` property also accepts an asterisk (*) as a wildcard to indicate that all Actuator endpoints should be exposed, as shown here:

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
```

If you want to expose all but a few endpoints, it's typically easier to include them all with a wildcard and then explicitly exclude a few. For example, to expose all Actuator endpoints except for `/threaddump` and `/heapdump`, you could set both the `management.endpoints.web.exposure.include` and `management.endpoints.web.exposure.exclude` properties like this:

```
management:
  endpoints:
    web:
      exposure:
        include: '*'
        exclude: threaddump,heapdump
```

Should you decide to expose more than `/health` and `/info`, it's probably a good idea to configure Spring Security to restrict access to the other endpoints. We'll look at

how to secure Actuator endpoints in section 15.4. For now, though, let's look at how you can consume the HTTP endpoints exposed by Actuator.

15.2 Consuming Actuator endpoints

Actuator can bestow a veritable treasure trove of interesting and useful information about a running application by way of the HTTP endpoints listed in table 15.1. As HTTP endpoints, these can be consumed like any REST API, using whatever HTTP client you wish, including Spring's `RestTemplate` and `WebClient`, from a JavaScript application, or simply with the `curl` command-line client.

For the sake of exploring Actuator endpoints, we'll use the `curl` command-line client in this chapter. In chapter 16, I'll introduce you to Spring Boot Admin, which layers a user-friendly web application on top of an application's Actuator endpoints.

To get some idea of what endpoints Actuator has to offer, a GET request to Actuator's base path will provide HATEOAS links for each of the endpoints. Using `curl` to make a request to `/actuator`, you might get a response something like this (abridged to save space):

```
$ curl localhost:8080/actuator
{
  "_links": {
    "self": {
      "href": "http://localhost:8080/actuator",
      "templated": false
    },
    "auditevents": {
      "href": "http://localhost:8080/actuator/auditevents",
      "templated": false
    },
    "beans": {
      "href": "http://localhost:8080/actuator/beans",
      "templated": false
    },
    "health": {
      "href": "http://localhost:8080/actuator/health",
      "templated": false
    },
    ...
  }
}
```

Because different libraries may contribute additional Actuator endpoints of their own, and because some endpoints may be not be exported, the actual results may vary from application to application.

In any event, the set of links returned from Actuator's base path serve as a map to all that Actuator has to offer. Let's begin our exploration of the Actuator landscape with the two endpoints that provide essential information about an application: the `/health` and `/info` endpoints.

15.2.1 Fetching essential application information

At the beginning of a typical visit to the doctor, we're usually asked two very basic questions: who are you and how do you feel? Although the words chosen by the doctor or nurse may be different, they ultimately want to know a little bit about the person they're treating and why you're seeing them.

Those same essential questions are what Actuator's `/info` and `/health` endpoints answer for a Spring Boot application. The `/info` endpoint tells you a little about the application, and the `/health` endpoint tells you how healthy the application is.

ASKING FOR INFORMATION ABOUT AN APPLICATION

To learn a little bit of information about a running Spring Boot application, you can ask the `/info` endpoint. By default, however, the `/info` endpoint isn't very informative. Here's what you might see when you make a request for it using `curl`:

```
$ curl localhost:8080/actuator/info
{}
```

Although it may seem that the `/info` endpoint isn't very useful, it's best to think of it as a clean canvas on which you may paint any information you'd like to present.

We have several ways to supply information for the `/info` endpoint to return, but the most straightforward way is to create one or more configuration properties where the property name is prefixed with `info`. For example, suppose that you want the response from the `/info` endpoint to include support contact information, including an email address and phone number. To do that, you can configure the following properties in the `application.yml` file:

```
info:
  contact:
    email: support@tacocloud.com
    phone: 822-625-6831
```

Neither the `info.contact.email` property nor the `info.contact.phone` property has any special meaning to Spring Boot or any bean that may be in the application context. However, by virtue of the fact that it's prefixed with *info*, the `/info` endpoint will now echo the value of the property in its response as follows:

```
{
  "contact": {
    "email": "support@tacocloud.com",
    "phone": "822-625-6831"
  }
}
```

In section 15.3.1, we'll look at a few other ways to populate the `/info` endpoint with useful information about an application.

INSPECTING APPLICATION HEALTH

Issuing an HTTP GET request for the `/health` endpoint results in a simple JSON response with the health status of your application. For example, here's what you might see when using `curl` to fetch the `/health` endpoint:

```
$ curl localhost:8080/actuator/health
{"status":"UP"}
```

You may be wondering how useful it is to have an endpoint that reports that the application is UP. What would it report if the application were down?

As it turns out, the status shown here is an aggregate status of one or more health indicators. Health indicators report the health of external systems that the application interacts with, such as databases, message brokers, and even Spring Cloud components such as Eureka and the Config Server. The health status of each indicator could be one of the following:

- *UP*—The external system is up and is reachable.
- *DOWN*—The external system is down or unreachable.
- *UNKNOWN*—The status of the external system is unclear.
- *OUT_OF_SERVICE*—The external system is reachable but is currently unavailable.

The health statuses of all health indicators are then aggregated into the application's overall health status, applying the following rules:

- If all health indicators are *UP*, then the application health status is *UP*.
- If one or more health indicators are *DOWN*, then the application health status is *DOWN*.
- If one or more health indicators are *OUT_OF_SERVICE*, then the application health status is *OUT_OF_SERVICE*.
- *UNKNOWN* health statuses are ignored and aren't rolled into the application's aggregate health.

By default, only the aggregate status is returned in response to a request for `/health`. You can configure the `management.endpoint.health.show-details` property, however, to show the full details of all health indicators, as shown next:

```
management:
  endpoint:
    health:
      show-details: always
```

The `management.endpoint.health.show-details` property defaults to `never`, but it can also be set to `always` to always show the full details of all health indicators, or to `when-authorized` to show the full details only when the requesting client is fully authorized.